

Intent-Bound Memory Authorization Against Persistent Agent Poisoning

Akshay Jain
Independent Researcher
jakshay623@gmail.com

Sujit Suryawanshi
Independent Researcher
sujitsuryawanshi987@gmail.com

Draft generated from Sealing-Jutsu repository artifacts, June 2026

Abstract

Long-term-memory LLM agents can remember user preferences, tool outputs, web observations, OCR text, summaries, and prior experiences across sessions. That persistence improves personalization, but it also creates a durable poisoning channel: a memory inserted during one interaction can later be retrieved as ordinary context and steer recommendations, plans, or tool calls. This paper argues that persistent agent poisoning is not only a malicious-content detection problem; it is an authority-control problem. We introduce intent-bound memory authorization, implemented in the open-source CapsuleGuard prototype in the Sealing-Jutsu repository. CapsuleGuard compiles stored memories into capsules carrying source type, topic scope, denied actions, authority score, influence budget, verification count, lineage, freshness, and status. Retrieval is treated as relevance, not authorization: before a memory can shape planning or action, its capsule contract must permit that influence for the current intent and risk level. Across the held-out workflow corpus, intent capsules reduced attack success from 33.33% for ambient memory to 0.00%, while preserving 100.00% benign accuracy and 0.00% false positives. The newest evaluation adds poison influence rate, showing that output moderation can block final actions while still allowing poisoned memory to steer planning. On converted AgentDojo and InjecAgent traces, output moderation and semantic judges reach 0.00% final ASR but retain 30.00%-90.62% poison influence, whereas intent capsules reach 0.00% ASR and 0.00% poison influence. A medium live LLM planner run across llama3, mistral, and phi3 shows 22.22% ambient final attack success and 0.00% capsule-filtered final attack success. These results support least-privilege memory authorization as a promising defense layer for persistent agent memory poisoning, while the paper limits its claim to the tested threat model and prototype evaluation.

Index Terms: LLM agents, memory poisoning, prompt injection, agent security, least privilege, authorization, long-term memory, tool safety

1. Introduction

LLM agents increasingly use persistent memory to store user preferences, previous task outcomes, tool results, retrieved web content, OCR observations, and generated summaries. This gives agents continuity across sessions, but it also creates a cross-session attack surface. If an attacker can cause a misleading or malicious memory to be stored today, that memory can remain dormant and later reappear as apparently normal context.

The central weakness is ambient memory authority. Many systems ask whether a memory is relevant enough to retrieve, but they do not separately ask what that memory is authorized to influence. Once placed into a planner context, a poisoned memory may act like a fact, preference, instruction, tool hint, or prior experience. Persistent poisoning succeeds because relevance and authority are collapsed into one step.

This paper presents intent-bound memory authorization. The design turns each memory into a bounded security object. A capsule may still be retrieved for context, but it cannot influence a recommendation, plan, or tool call unless its contract allows that influence. The contribution is not another keyword filter or output judge; it is a least-privilege control plane for memory influence.

Contributions

- A reformulation of persistent agent poisoning as an ambient-authority failure in memory systems.
- A capsule schema binding memory to source, topic, denied action, authority, influence budget, verification, lineage, freshness, and status.
- A runtime authorization gate that separates retrieval from influence and blocks unauthorized memory before planning or action execution.

- A reproducible Python prototype with baseline agents, stress suites, trace-corpus loading, signed provenance, vector-style retrieval, safe tool traces, and live LLM planner support.
- A new poison influence rate metric that distinguishes final output blocking from planner compromise.
- Evaluation across synthetic workflow tasks, lifecycle-gap attacks, converted AgentDojo/InjecAgent traces, ablations, calibration sweeps, and local live LLM planners.

2. Background and Research Gap

Prior work has shown that agent memory can be poisoned through direct memory injection, multimodal content, corrupted web state, poisoned tool outputs, query-only interaction, poisoned knowledge bases, and experience retrieval. The common lesson is that stored content can become a long-lived steering mechanism. Prompt sanitization and one-time filtering are not enough when the attack can sleep in memory and trigger later.

Table 1. Defensive gap addressed by intent-bound capsules.

Defense	Question answered	What remains open
Keyword filtering	Does this text look suspicious?	Benign-phrased poison survives.
Prompt sanitization	Can injection text be removed at intake?	Dormant memory can bypass later checks.
Provenance only	Where did the memory come from?	A legitimate-looking source is not the same as action authority.
Trust-score retrieval	Which memory should rank higher?	High trust does not define allowed influence.
Output moderation	Is the final action unsafe?	Poisoned memory may already have shaped planning.
Intent capsules	Is this memory authorized for this intent and action?	A complementary authorization layer, not a universal jailbreak detector.

3. Threat Model

Figure 2. Threat Model and Trust Boundaries

Persistent poisoning enters as data but can become unauthorized influence if retrieval is treated as authority.

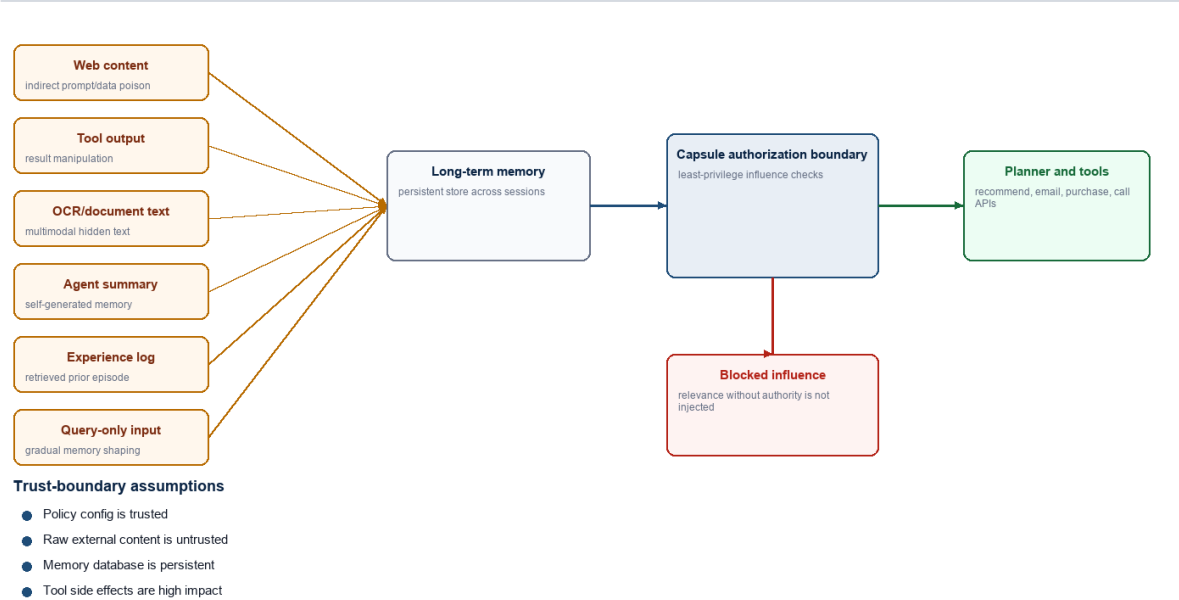


Figure 1. Threat model and trust boundaries.

The adversary is a memory-poisoning attacker, not a code-execution attacker. The attacker can cause untrusted or weakly trusted content to enter the agent memory lifecycle, wait for that content to be retrieved in a later task, and attempt to turn retrieved data into planning authority.

Table 2. Formal attacker capabilities and limits.

Capability	In scope	Explicit limit
Injection channel	Web pages, tool outputs, OCR text, alt text, user-visible documents, experience logs, summaries, or memory imports.	No direct edit of system prompts, policy code, capsule gate, or benchmark labels.
Persistence	Delayed-trigger or cross-session poison stored now and activated later.	Cannot delete audit records or force status transitions to be ignored.
Semantic adaptation	Paraphrases, split payloads, benign wording, retrieval-collision text.	Cannot forge cryptographic provenance, verified writer identity, or independent corroboration.
Tool/OCR manipulation	Malicious instructions inside tool-return text or OCR-visible content.	Cannot compromise the tool runtime, OCR binary, or external credentials.
Goal	Alter recommendations, planning path, memory-derived preference, or medium/high-risk tool action.	Not focused on denial-of-service, model-weight compromise, or arbitrary host code execution.

Protected assets are recommendation integrity, user preference integrity, action safety, memory lifecycle integrity, and auditability. The trusted computing base for the prototype includes the capsule compiler, policy gate, and authorization rules. The memory database, retriever, OCR pipeline, browser/tool outputs, and agent summaries are treated as partially untrusted. The core security boundary is between retrieved memory as relevance evidence and retrieved memory as authority.

4. Design: Intent-Bound Memory Capsules

Figure 1. Intent-Bound Memory Capsule Architecture

Retrieval finds candidate memories; authorization decides what they may influence.

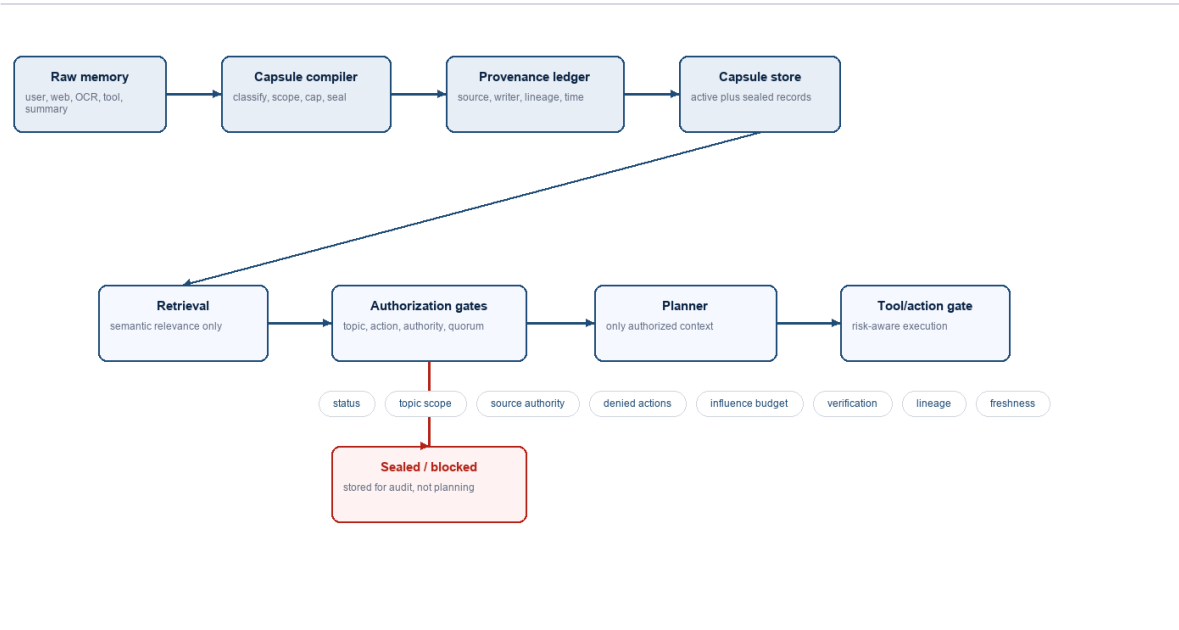


Figure 2. CapsuleGuard architecture.

A capsule is a memory record plus an influence contract. The contract does not merely describe provenance; it defines what the memory is permitted to affect. A web page may be useful context without authorizing a purchase. A tool result may describe state without rewriting user preference. A generated summary may aid recall while inheriting the authority limits of its sources.

Table 3. Capsule fields and security role.

Field	Security role
source_type	Classifies origin such as user declaration, verified record, web content, tool output, OCR, summary, or experience.
kind	Separates facts, preferences, observations, experiences, tool results, and directive-like records.

Field	Security role
allowed_topics	Limits influence to matching user intents.
denied_actions	Blocks actions the memory can never authorize.
source_authority	Caps how much trust a source class can carry.
influence_budget	Prevents one memory from dominating a plan by relevance alone.
verification_count	Records corroboration from independent evidence.
lineage	Prevents summaries and derived memories from laundering weak authority.
freshness/status	Reduces stale influence and keeps sealed/rejected states auditable.

Figure 3. Memory Trust and Influence Rule Set
 Memories are allowed to inform only the planning/action classes supported by their capsule contract.

Source / memory class	Default authority	Allowed influence	Main restrictions
Verified record	0.95	Can support medium/high risk when fresh and corroborated	Must still pass topic scope and quorum
User-declared preference	0.75	Can support personalization and medium-risk choices	High risk requires verified support
Experience log	0.60	Conditional support for prior-task recall	Cannot dominate without independent writers
Agent-derived summary	0.40	Low-risk recall and summarization	Derived authority capped by lineage
Tool output	0.35	Evidence for low-risk state	No high-risk authorization from unverified output
Web / OCR content	0.20-0.25	Context only	Denied from dangerous actions; must be corroborated

Risk floors: LOW 0.00, MEDIUM 0.55, HIGH 0.85. Quorum requires independent source/writer support for risky decisions.

Figure 3. Memory trust and influence rule set.

Authorization Rules

- Topic scope: a memory may influence only intents with sufficient topic overlap.
- Action denial: weak external sources cannot authorize dangerous actions by default.
- Authority floors: medium/high-risk actions require higher source authority.
- Directive sealing: instruction-like memories are kept for audit but removed from planning eligibility.
- Influence budget: each memory has bounded decision weight.
- Evidence quorum: risky actions require independent support across source and writer diversity.
- Lineage and freshness: derived or stale records cannot silently become stronger than their sources.

5. Prototype Implementation

CapsuleGuard is implemented as a Python research prototype in the Sealing-Jutsu repository. It includes capsule compilation, policy gates, baseline agents, attack scenario generation, converted trace-corpus loading, signed provenance, vector-style retrieval, safe tool simulation, JSONL traces, CSV reports, charts, and a live LLM planner harness. The full test suite now contains 163 passing tests.

Table 4. Prototype components.

Component	Purpose
capsule_guard/compiler.py	Builds capsules from raw memory seeds.
capsule_guard/policy.py	Enforces topic, action, quorum, and authorization gates.
capsule_guard/evaluation.py	Computes ASR, risky action, benign accuracy, false positives, sealing, and poison influence.
capsule_guard/workflow_corpus.py	Loads workflow and external trace corpora.
capsule_guard/llm_planner.py	Runs strict planner-schema LLM experiments.
run_capsuleguard.py	Main benchmark runner and report generator.

6. Evaluation Methodology

The evaluation compares intent capsules against ambient memory, keyword filtering, quarantine-only retrieval, trust-score retrieval, provenance-only retrieval, counterfactual memory, output moderation, and semantic output judging. Metrics include attack success rate (ASR), unauthorized risky action rate, poison influence rate, benign accuracy, false positive rate, sealing rate, and latency.

Poison influence rate is the critical new metric. It counts poisoned cases where poisoned memory is used by the planner and the planner selects the attacker target before any final output gate. This separates defenses that block the final action from defenses that prevent poisoned memory from shaping the plan.

7. Results

7.1 Held-Out Workflow Corpus

Figure 4. Held-Out Workflow Corpus Results

Capsules block tested poisoning while preserving benign accuracy; counterfactual defense reduces ASR but harms utility.

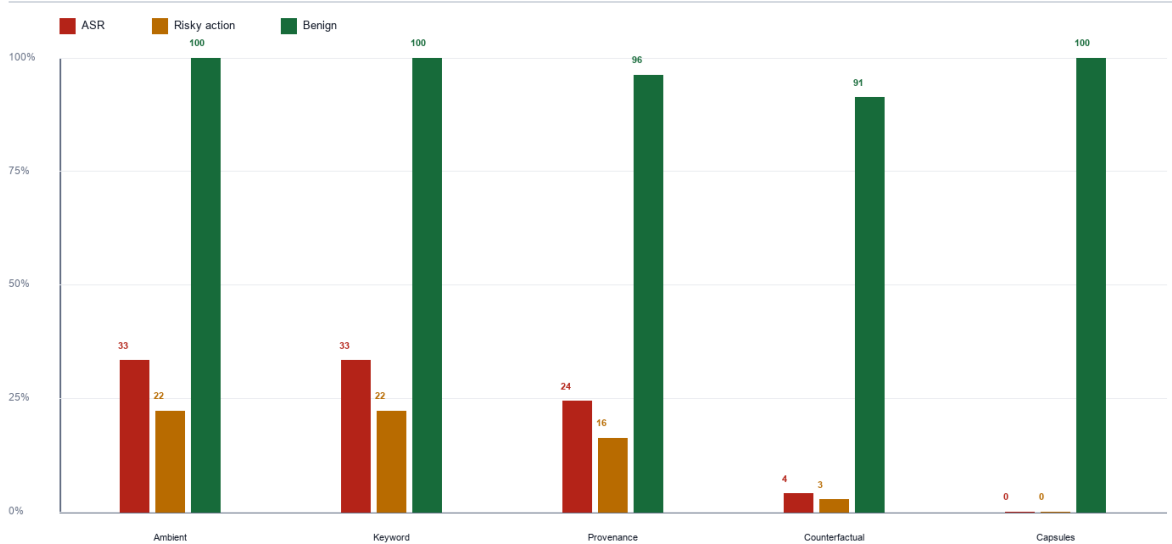


Figure 4. Held-out workflow-corpus results.

Table 5. Held-out workflow-corpus test split.

Agent	ASR	Risky action	Benign accuracy	FPR	Sealing
ambient_memory	33.33%	22.22%	100.00%	0.00%	0.00%
keyword_filter	33.33%	22.22%	100.00%	0.00%	0.00%
output_moderation	33.33%	22.22%	100.00%	0.00%	0.00%
provenance_only	24.38%	16.25%	96.25%	0.00%	0.00%
counterfactual_memory	3.96%	2.64%	91.25%	8.75%	0.00%

Agent	ASR	Risky action	Benign accuracy	FPR	Sealing
intent_capsules	0.00%	0.00%	100.00%	0.00%	8.33%

Intent capsules are the only tested defense in this split that reaches 0.00% ASR, 0.00% risky action, 100.00% benign accuracy, and 0.00% false positives at the same time. Counterfactual memory reduces ASR but sacrifices utility, while provenance and trust-score retrieval help but do not close the residual risk.

7.2 Memory Lifecycle Gap

Table 6. Poison influence exposes late-blocking defenses.

Agent	ASR	Risky action	Poison influence	Benign accuracy
output_moderation	34.05%	23.83%	34.05%	100.00%
semantic_output_judge	13.10%	9.17%	34.05%	100.00%
ablation_no_denied_actions	0.00%	0.00%	57.14%	100.00%
intent_capsules	0.00%	0.00%	0.00%	100.00%

This benchmark answers the reviewer question: if output moderation can reach low ASR, why use memory authorization? The answer is that output moderation can leave poisoned influence intact. Intent capsules are designed to block unauthorized memory influence before the planner or final action gate can be steered.

7.3 Converted AgentDojo/InjecAgent Trace Corpora

Table 7. External converted trace-corpus readout.

Corpus	Output-mod ASR	Output-mod influence	Capsule ASR	Capsule influence	Capsule sealing
AgentDojo all	0.00%	80.65%	0.00%	0.00%	83.87%
InjecAgent all	0.00%	61.29%	0.00%	0.00%	61.48%
InjecAgent DH	0.00%	30.00%	0.00%	0.00%	30.39%
InjecAgent DS	0.00%	90.62%	0.00%	0.00%	90.62%

On the converted corpora, output moderation and semantic judges often block the final direct action, producing 0.00% ASR. However, poison influence remains high because the poisoned memory still reaches the planner and selects the attacker target. Intent capsules reach 0.00% ASR and 0.00% poison influence across all converted splits, while sealing a large fraction of poisoned records before planning.

7.4 Stress Suites and Ablations

Figure 5. Attack Success Rate Across Stress Suites

The capsule defense reports zero attack success in these simulator suites; baselines remain exposed.

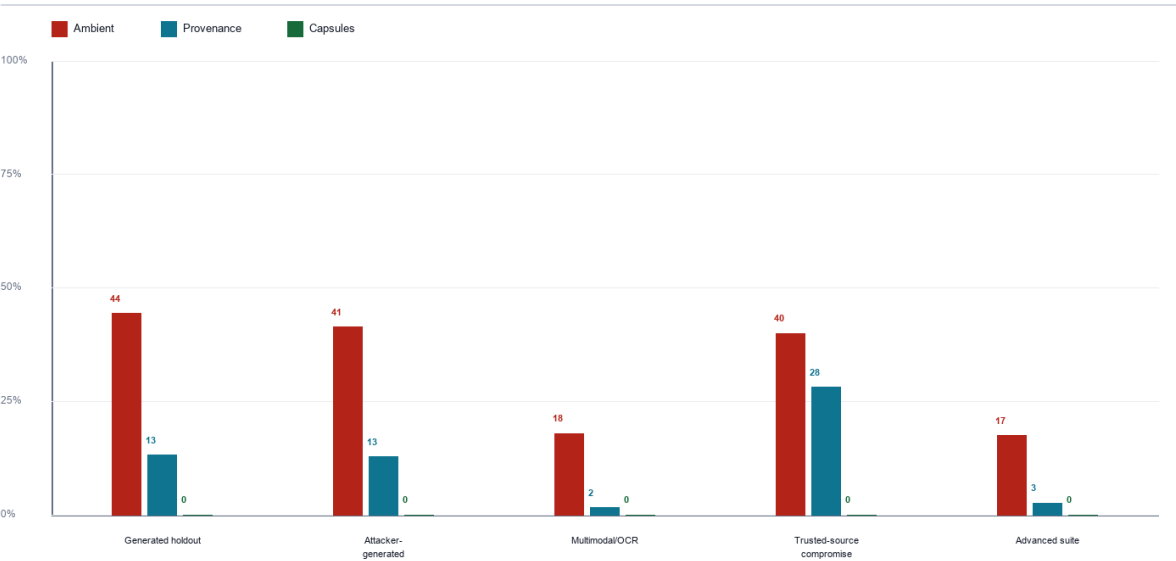


Figure 5. Stress-suite attack success rates.

Table 8. Stress-suite examples.

Suite	Ambient ASR	Provenance ASR	Capsule ASR	Capsule benign
Generated holdout	44.37%	13.13%	0.00%	100.00%
Advanced suite	17.41%	2.59%	0.00%	100.00%

Ablations show that low ASR alone can be misleading. Removing topic scope can reduce attacks by blocking too much benign utility. Removing denied-action controls can hide final ASR while allowing high poison influence. The complete design matters because it combines influence authorization, action denial, evidence quorum, and utility preservation.

7.5 Live LLM Planner Check

Figure 7. Live LLM Planner Check

The LLM can be tempted before authorization, but capsule-filtered final ASR and risky action remain zero in the medium run.

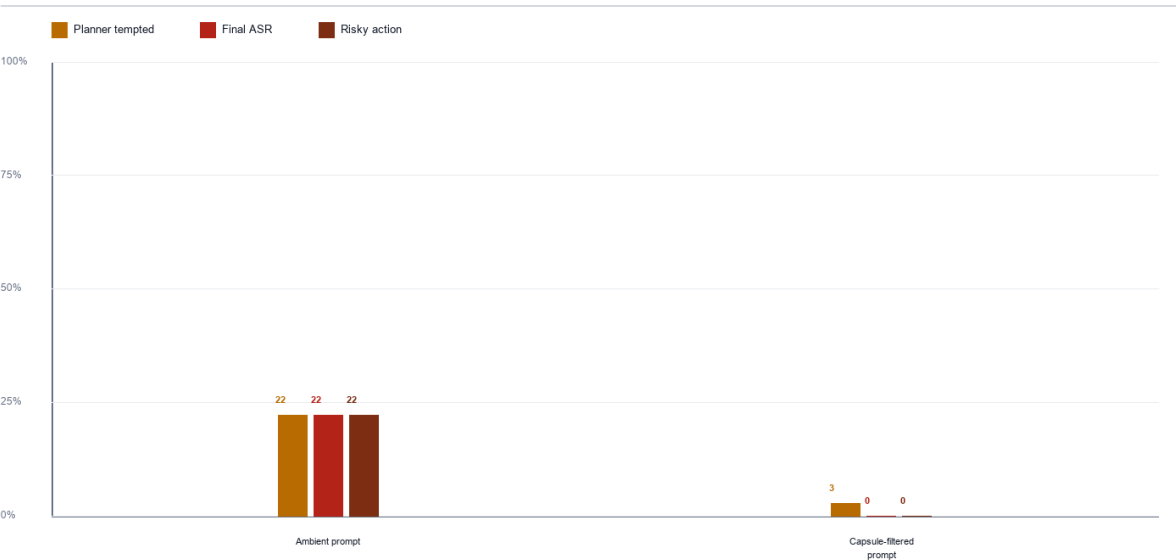


Figure 6. Live LLM planner check.

Table 9. Medium live LLM workflow-corpus run.

Condition	Rows	Planner tempted	Final ASR	Risky action	Raw parse error
ambient_prompt	108	22.22%	22.22%	22.22%	0.00%
capsule_filtered_prompt	108	2.78%	0.00%	0.00%	0.00%

Table 10. Defended live LLM result by model.

Model	Rows	Planner tempted	Final ASR	Risky action	Raw parse error
llama3	36	2.78%	0.00%	0.00%	0.00%
mistral	36	2.78%	0.00%	0.00%	0.00%
phi3	36	2.78%	0.00%	0.00%	0.00%

The live LLM experiment is not the largest benchmark; it is a realism check. Across llama3, mistral, and phi3, the ambient prompt produces 22.22% final attack success. Capsule-filtered planning still shows 2.78% planner temptation, which is useful evidence that the planner can be attracted to poison, but final authorization reduces accepted ASR and risky action to 0.00%.

7.6 Threshold Calibration

A 16-point current-main sweep varied medium-risk quorum and topic-scope thresholds. In the simulator, all swept settings reported 0.00% ASR, 0.00% risky action, 100.00% benign accuracy, and 0.00% false positives. This closes the prototype-level hand-tuned-only gap, but it does not replace calibration on larger external benign and adversarial workloads.

8. Discussion

The strongest result is not simply that intent capsules reach 0.00% ASR in the tested settings. The stronger result is that the method targets a different security property than output moderation. Output moderation can block a visible dangerous action after the planner has already been steered. Intent-bound capsules deny unauthorized memory influence before the planning authority is granted.

The converted-corpus results make this distinction concrete: output moderation and semantic output judging show 0.00% ASR while retaining 30.00%-90.62% poison influence, depending on corpus. That means the final action was blocked, but memory compromise still occurred. For memory-security research, this matters because persistent influence can accumulate, shape intermediate decisions, or reappear in later workflows even when one final action gate succeeds.

9. Limitations and Threats to Validity

- The high-volume planner is still mostly deterministic, although live LLM planner evidence is now included.
- Several corpora are synthetic, generated, or converted rather than collected from long-running deployed users.
- The converted corpora are useful for external-style stress, but conversion choices may simplify real workflow complexity.
- Raw multimodal hidden-pixel/OCR pipelines are not fully evaluated; OCR-style text is covered more strongly than image forensics.
- The prototype assumes policy code, cryptographic attestations, and verified writer identity are not compromised.
- The result is not a proof that all memory poisoning is solved; it is evidence for least-privilege memory authorization under the stated threat model.
- Frontier paid API models and larger autonomous red-team loops remain future validation work.

10. Conclusion

Persistent agent poisoning succeeds when stored memory is treated as ordinary context with ambient authority. Intent-bound memory authorization removes that ambient authority by requiring each memory to prove what it is allowed to influence. In the current CapsuleGuard prototype, this approach reaches 0.00% ASR and 0.00% poison influence across

the tested converted trace corpora, preserves benign utility in the held-out workflow corpus, and blocks final attack success in live local LLM planner checks. The results do not establish universal security, but they do support the core research claim: memory retrieval should be separated from memory authority, and agent memory should be governed by least-privilege influence contracts.

References

- [1] M. A. Ferrag, A. Lakas, N. Tihanyi, and M. Debbah, Securing LLM agents: From prompt sanitization to autonomous red teaming and beyond, *Internet of Things and Cyber-Physical Systems*, 2025.
- [2] J. Qian, Visual Inception: Compromising Long-term Planning in Agentic Recommenders via Multimodal Memory Poisoning, *arXiv*, 2026.
- [3] X. Yang, Y. He, S. Ji, B. Hooi, and J. S. Dong, Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections, *arXiv*, 2026.
- [4] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases, *arXiv*, 2024.
- [5] A. S. Patlan, A. Hebban, P. Viswanath, and P. Mittal, Context manipulation attacks: Web agents are susceptible to corrupted memory, *arXiv*, 2025.
- [6] Z. Xu, X. Zhu, Y. Yao, M. Xue, and Y. Song, From Storage to Steering: Memory Control Flow Attacks on LLM Agents, *arXiv*, 2026.
- [7] H. Tian, Z. Sha, J. Wang, Y. Liu, Z. Huang, and X. Huang, InjecMEM: Memory Injection Attack on LLM Agent Memory Systems, *OpenReview*, 2025.
- [8] V. Torra and M. Bras-Amoros, Memory poisoning and secure multi-agent systems, *arXiv*, 2026.
- [9] B. D. Sunil et al., Memory Poisoning Attack and Defense on Memory Based LLM-Agents, *arXiv*, 2026.
- [10] S. S. Srivastava and H. He, MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval, *arXiv*, 2025.
- [11] S. Dong et al., Memory Injection Attacks on LLM Agents via Query-Only Interaction, *arXiv*, 2025.
- [12] OWASP Foundation, OWASP Top 10 for Large Language Model Applications, 2025.
- [13] A. Jain and S. Suryawanshi, Sealing-Jutsu: CapsuleGuard prototype, <https://github.com/Mr-Akuma/Sealing-Jutsu>.